



Data Wrangling: Clean, Transform, Merge, Reshape

Python for Data Analysis - Cap. 8

Indexação Hierárquica

- Permite múltiplos níveis de indexação em um mesmo eixo;
- Útil para agrupamentos de dados;
- Permite seleção de índices específicos.

```
In [14]: data["b"]
```

```
Out[14]:
```

```
1    0.204560
```

```
3    0.567725
```

```
dtype: float64
```

```
In [15]: data["b":"c"]
```

```
Out[15]:
```

```
b 1    0.204560
```

```
3    0.567725
```

```
c 1    0.595545
```

```
2    0.964515
```

```
dtype: float64
```

```
In [16]: data.loc[["b", "d"]]
```

```
Out[16]:
```

```
b 1    0.204560
```

```
3    0.567725
```

```
d 2    0.653177
```

```
3    0.748907
```

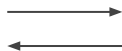
```
dtype: float64
```

Indexação Hierárquica

- É possível rearranjar os dados, mudando a sua exibição por meio de funções como `stack()` e `unstack()`.

```
In [18]: data.unstack()
Out[18]:
```

	1	2	3
a	0.929616	0.316376	0.183919
b	0.204560	NaN	0.567725
c	0.595545	0.964515	NaN
d	NaN	0.653177	0.748907



```
In [19]: data.unstack().stack()
Out[19]:
```

a	1	0.929616
	2	0.316376
	3	0.183919
b	1	0.204560
	3	0.567725
c	1	0.595545
	2	0.964515
d	2	0.653177
	3	0.748907

dtype: float64

Indexação Hierárquica

```
In [24]: frame
```

```
Out[24]:
```

state		Ohio		Colorado
color		Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```
In [25]: frame.index.nlevels
```

```
Out[25]: 2
```

```
In [20]: frame = pd.DataFrame(np.arange(12).reshape((4, 3)),
```

```
.....:                        index=[["a", "a", "b", "b"], [1, 2, 1, 2]],
.....:                        columns=[["Ohio", "Ohio", "Colorado"],
.....:                                ["Green", "Red", "Green"]])
```

```
pd.MultiIndex.from_arrays([["Ohio", "Ohio", "Colorado"],
                             ["Green", "Red", "Green"]],
                           names=["state", "color"])
```

```
In [26]: frame["Ohio"]
```

```
Out[26]:
```

color		Green	Red
key1	key2		
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

```
In [29]: frame.swaplevel(0, 1).sort_index(level=0)
```

```
Out[29]:
```

state		Ohio		Colorado
color		Green	Red	Green
key2	key1			
1	a	0	1	2
	b	6	7	8
2	a	3	4	5
	b	9	10	11

Combinando e unindo *Datasets*

- *pandas.merge* - combina duas *Series* ou *DataFrames* com uma operação semelhante ao *join* em bases de dados;
 - *pandas.concat* - concatena duas ou mais *Series* ou *DataFrames* ao longo de um eixo específico (horizontal ou vertical);
 - *combine_first* - atualiza elementos nulos de um *DataFrame* com elementos não nulos de outro *DataFrame* no mesmo local;
- 

pandas.merge

```
In [38]: df1 = pd.DataFrame({"key": ["b", "b", "a", "c", "a", "a", "b"],  
.....:                      "data1": pd.Series(range(7), dtype="Int64")})
```

```
In [39]: df2 = pd.DataFrame({"key": ["a", "b", "d"],  
.....:                      "data2": pd.Series(range(3), dtype="Int64")})
```

```
In [40]: df1
```

```
Out[40]:
```

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

```
In [41]: df2
```

```
Out[41]:
```

	key	data2
0	a	0
1	b	1
2	d	2

Resultado:

```
In [42]: pd.merge(df1, df2)
```

```
Out[42]:
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	b	6	1
3	a	2	0
4	a	4	0
5	a	5	0

Table 8-1. Different join types with the how argument

Option	Behavior
how="inner"	Use only the key combinations observed in both tables
how="left"	Use all key combinations found in the left table
how="right"	Use all key combinations found in the right table
how="outer"	Use all key combinations observed in both tables together

```
In [47]: pd.merge(df1, df2, how="outer")
```

```
Out[47]:
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	b	6	1
3	a	2	0
4	a	4	0
5	a	5	0
6	c	3	<NA>
7	d	<NA>	2

pandas.concat

```
In [61]: s1 = Series([0, 1], index=['a', 'b'])
```

```
In [62]: s2 = Series([2, 3, 4], index=['c', 'd', 'e'])
```

```
In [63]: s3 = Series([5, 6], index=['f', 'g'])
```

```
In [68]: pd.concat([s1, s4], axis=1, join='inner')
```

```
Out[68]:
```

	0	1
a	0	0
b	1	5

```
In [64]: pd.concat([s1, s2, s3])
```

```
Out[64]:
```

a	0
b	1
c	2
d	3
e	4
f	5
g	6

```
In [65]: pd.concat([s1, s2, s3], axis=1)
```

```
Out[65]:
```

	0	1	2
a	0	NaN	NaN
b	1	NaN	NaN
c	NaN	2	NaN
d	NaN	3	NaN
e	NaN	4	NaN
f	NaN	NaN	5
g	NaN	NaN	6

combine_first

```
In [84]: a = Series([np.nan, 2.5, np.nan, 3.5, 4.5, np.nan],  
.....:             index=['f', 'e', 'd', 'c', 'b', 'a'])
```

```
In [85]: b = Series(np.arange(len(a), dtype=np.float64),  
.....:             index=['f', 'e', 'd', 'c', 'b', 'a'])
```

```
In [86]: b[-1] = np.nan
```

```
In [87]: a
```

```
Out[87]:
```

```
f    NaN  
e    2.5  
d    NaN  
c    3.5  
b    4.5  
a    NaN
```

```
In [88]: b
```

```
Out[88]:
```

```
f     0  
e     1  
d     2  
c     3  
b     4  
a    NaN
```

```
In [89]: np.where(pd.isnull(a), b, a)
```

```
Out[89]:
```

```
f    0.0  
e    2.5  
d    2.0  
c    3.5  
b    4.5  
a    NaN
```

```
In [90]: b[:-2].combine_first(a[2:])
```

```
Out[90]:
```

```
a    NaN  
b    4.5  
c    3.0  
d    2.0  
e    1.0  
f    0.0
```


Reestruturação e Pivotamento

- Transformações entre formatos “*long*” e “*wide*”;
- `stack()` e `unstack()` convertem níveis de índice;
- `pivot()` reorganiza colunas em formato largo.



Pivotamento “*long*” para “*wide*”

```
In [116]: ldata[:10]
```

```
Out[116]:
```

	date	item	value
0	1959-03-31 00:00:00	realgdp	2710.349
1	1959-03-31 00:00:00	infl	0.000
2	1959-03-31 00:00:00	unemp	5.800
3	1959-06-30 00:00:00	realgdp	2778.801
4	1959-06-30 00:00:00	infl	2.340
5	1959-06-30 00:00:00	unemp	5.100
6	1959-09-30 00:00:00	realgdp	2775.488
7	1959-09-30 00:00:00	infl	2.740
8	1959-09-30 00:00:00	unemp	5.300
9	1959-12-31 00:00:00	realgdp	2785.204



```
In [117]: pivoted = ldata.pivot('date', 'item', 'value')
```

```
In [118]: pivoted.head()
```

```
Out[118]:
```

item	infl	realgdp	unemp
date			
1959-03-31	0.00	2710.349	5.8
1959-06-30	2.34	2778.801	5.1
1959-09-30	2.74	2775.488	5.3
1959-12-31	0.27	2785.204	5.6
1960-03-31	2.31	2847.699	5.2

Pivotamento “*wide*” para “*long*”

Melt

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



df3.melt(id_vars=['first', 'last'])

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

Transformações de Dados

- Transformações comuns:
 - Remoção de duplicatas;
 - Mapeamento e substituição;
 - Renomeação de índices e colunas;
 - Discretização de dados.

